



ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR
Membre de
HONORIS UNITED UNIVERSITIES

Ecole Reconnue par l'Etat

Ecole Marocaine des Sciences de l'Ingénieur

EMSI — Casablanca

Digital Banking

Application de gestion bancaire securisee par JWT,
avec assistant intelligent RAG et client Telegram.

Spring Boot 3

Spring Security/JWT

JPA/Hibernate

Angular 17

Spring AI — RAG

Telegram Bot

Rapport de projet

Realise par

Filiere / Groupe

Encadre par

Annee universitaire

Digital Banking

KHATIB Nassime

4DSIM — G1-2

Pr. Mohamed YOUSSEFI

2025 / 2026

Sommaire

1. Introduction et contexte
2. Objectifs du projet
3. Architecture generale
4. Technologies utilisees
5. Partie 1 — Back-end : modele de donnees et API REST
6. Partie 2 — Securite : Spring Security et JWT
7. Partie 3 — Front-end Angular
8. Partie 4 — Chatbot RAG et client Telegram
9. Partie 5 — Integration du chatbot dans l'application
10. Fonctionnalites supplementaires
11. Installation et execution
12. Tests
13. Difficultes rencontrees et solutions
14. Conclusion et perspectives

1. Introduction et contexte

Ce projet a pour objectif la réalisation d'une application web full-stack de gestion bancaire numérique (Digital Banking). L'application permet de gérer les clients et leurs comptes, d'effectuer des opérations financières (versement, retrait, virement), de visualiser des statistiques et d'interroger un assistant conversationnel intelligent.

Le projet se compose de deux applications : une API REST Spring Boot 3 côté serveur et une SPA Angular 17 côté client, communiquant via HTTP sécurisée par jetons JWT.

2. Objectifs du projet

Le cahier des charges se décompose en cinq parties :

- **Partie 1 — Back-end** : entités JPA, repositories, couche service avec DTOs/mappers, API REST + Swagger.
- **Partie 2 — Sécurité** : Spring Security + JWT.
- **Partie 3 — Front-end** : SPA Angular consommant l'API.
- **Partie 4 — IA** : chatbot RAG (Retrieval-Augmented Generation) + client Telegram.
- **Partie 5 — Integration** : chatbot intégré dans l'application web.

Fonctionnalités supplémentaires : CRUD clients, gestion comptes/opérations, audit, gestion utilisateurs, tableau de bord graphique.

3. Architecture générale

Architecture 3-tiers : présentation Angular | logique Spring | persistance JPA.

```
CLIENT Angular 17
Composants standalone | HTTP Interceptor JWT | Route Guards | Chart.js

----- HTTP / JSON + JWT Bearer ----->

SERVEUR Spring Boot 3
REST Controllers | Services métier | Spring Data JPA
Spring Security/JWT | Spring AI RAG

----- JPA / Hibernate ----->

BASE DE DONNEES H2 (dev) / MySQL (prod)

TELEGRAM -- Bot API long-polling --> même service RAG
```

4. Technologies utilisées

Domaine	Technologie	Role
Langage serveur	Java 17	Developpement back-end
Framework serveur	Spring Boot 3.2.5	Conteneur et autoconfiguration
Persistence	Spring Data JPA / Hibernate	ORM et repositories
Base de donnees	H2 (dev) / MySQL (prod)	Stockage
Securite	Spring Security + JWT HS256	Auth / autorisation
Doc. API	springdoc (Swagger UI)	Documentation interactive
IA	Spring AI 1.0 + OpenAI	Chatbot RAG
Messagerie	Telegram Bot API	Client externe
Outils	Lombok, Maven	Productivite / build
Front-end	Angular 17 standalone	SPA
Graphiques	Chart.js / ng2-charts	Tableau de bord

5. Partie 1 — Back-end : modele de donnees et API REST

5.1 Modele de donnees

BankAccount (abstraite) est specialisee en **CurrentAccount** (overDraft) et **SavingAccount** (interestRate). Strategie SINGLE_TABLE avec colonne discriminante TYPE.

Entite	Description	Proprietes cles
Customer	Client	id, name, email, comptes, createdBy
BankAccount (abs.)	Compte generique	id UUID, balance, status, currency
CurrentAccount	Compte courant	overDraft
SavingAccount	Compte epargne	interestRate
AccountOperation	Operation	date, montant, type DEBIT/CREDIT
AppUser/AppRole	Utilisateurs et roles	username, password BCrypt, roles N-N

5.2 Repositories

Interfaces JpaRepository avec requetes derivees et JPQL : recherche clients par mot-cle, historique page des operations.

5.3 Service, DTOs et Mappers

BankAccountServiceImpl : CRUD clients, creation comptes, debit/credit/virement (exception BalanceNotSufficientException). Conversion entite ↔ DTO via BankAccountMapperImpl. Serialisation polymorphe Jackson @JsonTypeInfo.

5.4 API REST et Swagger

Controleur	Prefixe	Operations
CustomerRestController	/customers	CRUD + recherche (suppression ADMIN)
BankAccountRestController	/accounts	Comptes, historique page, debit/credit/virement
DashboardRestController	/dashboard	Statistiques
UserRestController	/users	Utilisateurs et roles (ADMIN)

Swagger : <http://localhost:8085/swagger-ui.html>. Gestion globale des exceptions via @RestControllerAdvice.

6. Partie 2 — Securite : Spring Security et JWT

Resource Server OAuth2 stateless — JWT signes HMAC HS256.

- **Login** : /auth/login verifie les identifiants, genere un JWT avec claim scope = roles.
- **Mots de passe** : BCryptPasswordEncoder.
- **Autorisation** : @EnableMethodSecurity + @PreAuthorize sur actions ADMIN.
- **Endpoints publics** : login, inscription, Swagger, console H2.
- **CORS** : autorise localhost:4200.
- **Gestion comptes** : inscription, changement mot de passe, attribution de roles.

Securite : secret JWT et cles API fournis via variables d'environnement uniquement.

7. Partie 3 — Front-end Angular

Angular 17 avec composants standalone (sans NgModule), organise en features et services core.

Module	Role
Authentification	Connexion, changement mot de passe
Clients	Liste, recherche, ajout/modification
Comptes	Detail, historique pagine, debit/credit/virement
Tableau de bord	Graphiques Chart.js : types de comptes, operations, soldes
Chatbot	Widget flottant integre

Points techniques

- Intercepteur HTTP ajoutant automatiquement le token Bearer (sauf /auth/login).
- Route Guard protegeant les pages authentifiees.
- Decodage JWT cote client pour affichage conditionnel des actions ADMIN.
- ng2-charts / Chart.js pour les visualisations.

8. Partie 4 — Chatbot RAG et client Telegram

8.1 Principe du RAG

Trois etapes : indexation, recherche semantique, generation augmentee.

- **Indexation** : KnowledgeBaseLoader vectorise la FAQ bancaire + donnees dynamiques via OpenAI text-embedding-3-small → SimpleVectorStore en memoire.
- **Recherche** : top-K documents les plus proches de la question (similarite cosinus).
- **Generation** : documents injectes dans le prompt → reponse via gpt-4o-mini.

Endpoint : **POST /chatbot/ask**.

8.2 Client Telegram

TelegramBotService interroge l'API en long-polling, transmet les messages au service RAG et retourne la reponse. S'active si un token @BotFather est fourni.

9. Partie 5 — Integration du chatbot dans l'application

Widget flottant Angular appelant /chatbot/ask : l'utilisateur interroge l'IA sans quitter sa page. Meme logique RAG accessible depuis le web et Telegram.

Module optionnel (app.chatbot.enabled) : l'application fonctionne sans cle OpenAI ; le chatbot s'active via un profil Spring dedie.

10. Fonctionnalités supplémentaires

Fonctionnalité	Description
Audit	Enregistrement de l'utilisateur authentifié sur chaque action (createdBy / performedBy depuis le SecurityContext).
CRUD clients	Creation, modification, suppression, recherche par mot-cle.
Gestion comptes	Comptes courant/epargne, operations, historique page.
Utilisateurs	Inscription, roles, changement mot de passe securise.
Tableau de bord	Statistiques visuelles : types comptes, volumes, soldes.
DataInitializer	Peuple la base au demarrage et sert de test integration manuel.

11. Installation et execution

11.1 Back-end

```
cd backend
mvn spring-boot:run # http://localhost:8085
```

Comptes de demonstration :

Utilisateur	Mot de passe	Role
admin	admin123	ADMIN
user1	user123	USER

Swagger : [/swagger-ui.html](#) | Console H2 : [/h2-console](#) (jdbc:h2:mem:bank).

11.2 Front-end

```
cd frontend
npm install
npm start # http://localhost:4200
```

11.3 Chatbot (optionnel)

```
$env:OPENAI_API_KEY="sk-..."
mvn spring-boot:run "-Dspring-boot.run.profiles=chatbot"
```

Securite : aucune cle dans le code source — variables d'environnement uniquement. .gitignore empeche la publication accidentelle.

12. Tests

Test couche DAO avec `@DataJpaTest` (`CustomerRepositoryTest`). `DataInitializer` joue le role de test d'integration manuel. Validation des endpoints via Swagger UI.

13. Difficultes rencontrees et solutions

Difficulte	Solution
Dependance circulaire securite / <code>UserDetailsService</code> .	<code>UserDetailsService</code> passe en parametre du bean <code>AuthenticationManager</code> (lazy).
Serialisation JSON polymorphe (courant vs epargne).	<code>@JsonTypeInfo</code> Jackson avec propriete discriminante type.
Demarrage impossible sans cle OpenAI.	<code>@ConditionalOnProperty</code> + exclusion auto-configuration OpenAI par default.
Quota OpenAI depasse (HTTP 429).	Credit prepaye requis. Application fonctionnelle sans chatbot.

14. Conclusion et perspectives

Projet full-stack complet : persistance JPA, API REST documentee, securite JWT, SPA Angular reactive et assistant RAG multi-canal (web + Telegram).

Pistes d'amelioration :

- Base vectorielle persistante : PgVector, Redis ou Chroma.
- Tests d'integration automatises sur les controleurs REST.
- Refresh tokens pour le renouvellement des sessions.
- Conteneurisation Docker Compose.